24. Packages: apt/dpkg (Debian/Ubuntu)

24. Packages: apt/dpkg (Debian/Ubuntu)

Phase: Phase 8 — Packages, disks, storage
Estimated time: 1–2 days
Slug: `24-packages-apt`

Learning objectives

- Update and install with apt
- Query packages with dpkg/apt
- Clean caches thoughtfully
- Pin/version awareness

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

On Debian and Ubuntu, software arrives as `.deb` packages managed by two layers: `dpkg`, which installs single files and tracks what is installed, and `apt`, which fetches from repositories and resolves dependencies. Servers need reproducible installs, security updates, and the ability to answer “which package owns this file” and “what did that upgrade change”.

A key point that trips beginners: `apt update` does not upgrade anything. It only refreshes the local package indexes (the list of available versions). `apt upgrade` is what installs newer versions. That is why the exercise answer is `refresh-indexes`.

Core concepts

Command	Effect
`apt update`	Refresh package indexes from repositories
`apt upgrade`	Install newer versions of installed packages
`apt full-upgrade`	Upgrade, allowing removals to resolve deps
`apt install pkg`	Install with dependencies
`apt remove pkg`	Remove, keep config
`apt purge pkg`	Remove including config
`apt autoremove`	Remove no-longer-needed dependencies
`apt search / show`	Find and describe packages
`apt-cache policy pkg`	Installed vs candidate version
`dpkg -l` / `dpkg -L pkg`	List packages / files of a package
`dpkg -S /path`	Which package owns a file
`dpkg -i file.deb`	Install a local file (no dependency fetch)

Where things live:

Path	Content
`/etc/apt/sources.list`, `/etc/apt/sources.list.d/`	Repository definitions
`/var/lib/apt/lists/`	Downloaded indexes
`/var/cache/apt/archives/`	Downloaded `.deb` files
`/var/lib/dpkg/status`	Installed package database

Layers and data flow:

```
repositories (archive.ubuntu.com, security.ubuntu.com)
      |
      |  apt update      (downloads Packages indexes only)
      v
/var/lib/apt/lists/      <- local catalog of what exists
      |
      |  apt install / upgrade   (resolve deps, download .deb)
      v
/var/cache/apt/archives/*.deb
      |
      |  dpkg (unpack, configure)
```

```
       v
  /var/lib/dpkg/status  +  files on disk
```

Worked examples

Refresh and see what could be upgraded:

```
sudo apt update
apt list --upgradable 2>/dev/null | head -4
```

Sample output:

```
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Reading package lists... Done
8 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

Install and inspect a package:

```
sudo apt install -y tree
apt-cache policy tree
dpkg -L tree | head -4
dpkg -S "$(command -v tree)"
```

Sample output:

```
tree:
  Installed: 2.1.1-2ubuntu3
  Candidate: 2.1.1-2ubuntu3
/usr/bin/tree
tree: /usr/bin/tree
```

Upgrade, then clean up:

```
sudo apt upgrade -y
sudo apt autoremove -y
sudo apt clean
```

Remove versus purge, and hold a version:

```
sudo apt remove nginx
sudo apt purge nginx
sudo apt-mark hold linux-image-generic
apt-mark showhold
```

Add a third-party repo carefully (key then source), then verify:

```
curl -fsSL https://example.com/key.gpg | sudo gpg --dearmor -o /etc/apt/keyrings/
example.gpg
echo "deb [signed-by=/etc/apt/keyrings/example.gpg] https://example.com/apt
stable main" | sudo tee /etc/apt/sources.list.d/example.list
sudo apt update
```

Helper from the exercise: `apt_update_does() { echo refresh-indexes; }`

Mini-lab

1. Run `sudo apt update` and read the Hit, Get, and Ign lines.
2. Run `apt list --upgradable`; note that nothing was installed yet.
3. Install `tree` and `jq`; use `which` and `dpkg -L jq` to see files.
4. Find the owner of `/bin/ls` with `dpkg -S /bin/ls`.
5. Compare `apt-cache policy` for a package before and after an upgrade.
6. Remove `jq` with `remove`, check config remains, then use `purge`.
7. Use `apt show tree` and read the Depends line.
8. Run `apt-mark hold tree`, try an upgrade, then `unhold`.

Decision guide / failure modes

Symptom	Cause	Fix
`Unable to locate package`	Stale indexes or wrong name	`apt update`, `apt search`
`Could not get lock`	Another apt or unattended-upgrades running	Wait; do not delete lock files blindly
`Hash Sum mismatch`	Mirror sync or proxy	Retry, `apt clean`, change mirror
`NO_PUBKEY` / signature error	Missing repo key	Install the key in `/etc/apt/keyrings`
Broken dependencies	Half-installed package	`sudo apt -f install`, `dpkg --configure -a`
Wrong version installed	Another repo has higher priority	`apt-cache policy`, pinning

Habits: run `apt update` before `apt install`, prefer `apt` over raw `dpkg -i`, keep unattended security updates on, and read the proposed removals before pressing yes.

Common mistakes

- disabling signature checks
- autoremoving blindly on prod
- mixing distro versions

Exercises

Work in `exercises/24-packages-apt/`. Read that folder's `README.md` fully before coding.

```
cd exercises/24-packages-apt
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement apt_update_does in exercise.sh. It takes no arguments and prints one hyphenated token describing what apt update does: it refreshes the package indexes (the list of available packages and versions) but installs nothing. The test compares the output exactly, so print only that token in lowercase. As a follow-up on a Debian or Ubuntu system, run sudo apt update and then apt list –upgradable to confirm that no package changed until you run apt upgrade.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/24-packages-apt/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. apt update vs upgrade?
2. How list files owned by a package?
3. Why official repos?

Further reading

- https://ubuntu.com/server/docs/package-management